

ENGINEERING

ServicePro Platform Assessment

1. Executive Summary

Publication edition | 2026-08-03

Document Control

Purpose	Enterprise platform overview and buyer evaluation reference
Audience	Business leaders, platform administrators, evaluators, partners, and technical stakeholders
Scope	Capabilities, architecture, security, deployment, adoption, outcomes, and terminology
Source	ServicePro repository documentation; technical meaning preserved

Version: 8.0.0-alpha.1 Assessment Date: July 2026 Prepared by: Engineering

ServicePro is a multi-tenant SaaS field service management platform built on Node.js (API) and Next.js 15 (Web). The platform currently runs on Render (free tier) with Supabase PostgreSQL and Cloudflare Workers for frontend delivery. It serves two active workspaces (tenant_demo / Aqua Pro Plumbing, cd_tenant_demo / C & D Landscaping) with a single platform admin (5189213@gmail.com).

The codebase contains 14,255 files spanning an ambitious multi-phase roadmap from alpha through v7 GA, with extensive documentation (551 sprint docs, 48 phase docs) but a much smaller operational core.

2. Repository Metrics

Metric	Count
Total files	14,255
API route modules	177
Repository modules	179
Service modules	161
Middleware modules	16
Frontend components (TSX)	44
PostgreSQL migrations	7
Test files	770
Seed/utility scripts	702
Sprint documentation files	551
Phase documentation files	48
GitHub Actions workflows	16

3. Architecture Overview

3.1 Runtime Stack

Layer	Technology	Notes
API Server	Node.js (raw HTTP, no Express)	apps/api/src/server.js
Web Frontend	Next.js 15.5.20, React 19.1.1	Static export for Cloudflare Workers

Layer	Technology	Notes
Database	PostgreSQL 16 (Supabase)	Tenant-per-schema isolation
Auth	JWT + bcryptjs	Custom auth middleware
Deployment (API)	Render (free tier)	Auto-deploy on main, cold start ~50s
Deployment (Web)	Cloudflare Workers	Via wrangler.toml
Domain	aardvark-enterprises.net	CORS configured for multiple origins

3.2 Monorepo Structure

```

ServiceRepo/
├── apps/
│   ├── api/           # Node.js HTTP API (production-active)
│   ├── web/           # Next.js frontend (production-active)
│   ├── admin/         # Platform admin UI (React, development)
│   ├── customer-portal/ # Customer-facing portal (development)
│   └── mobile/         # Mobile app (planned)
├── migrations/postgres/ # 7 SQL migration files
├── scripts/             # 702 seed, utility, and test runner scripts
├── tests/               # 770 test files
├── docs/               # 600+ documentation files
└── .github/workflows/  # 16 CI/CD workflows

```

3.3 Request Flow

- 1 Raw HTTP request → server.js → router.js
- 2 Security headers, CORS, rate limiting, body limit
- 3 Public routes (storefront, blog, auth) served without auth
- 4 Authenticated routes: JWT verification → tenant resolution → permission check
- 5 Route handler → service layer → repository → PostgreSQL

3.4 Multi-Tenancy Model

- Tenant ID extracted from JWT claims or x-tenant-id header
- Schema-per-tenant isolation in PostgreSQL
- Platform admin resolves across tenants via tenantResolver.js
- Cross-tenant login lookup enables owners to auth from any context

4. Operational Core vs. Planned Features

4.1 Production-Active Modules

The following modules are actively serving production traffic:

- Auth — login, register, refresh, password reset, MFA verify, invitation accept
- Customers — CRUD with delete, search
- Jobs — CRUD with status transitions
- Estimates & Invoices — CRUD, PDF generation

- Payments — processing and history
- Services & Pricebook — catalog management
- Technicians & Dispatch — scheduling, assignment
- Storefront Builder — theme, branding, service configuration
- Public Storefront — SEO pages, service request intake, blog, financing
- CRM — leads pipeline, marketing campaigns
- AI Assistant — chat integration
- Platform Admin — owner management, workspace CRUD, tenant management center
- Customer Portal — booking, invoice viewing
- Dashboard — operational metrics
- Reports & Exports — catalog, scheduling, export
- Workflows & Automation — rule engine, job transitions
- Notifications — event-driven messaging
- Inventory & Materials — stock tracking
- Audit & Security — event logging, rate limit visibility
- Observability — metrics, summary
- File Upload — asset management

4.2 Scaffolded-but-Inactive Modules (Phases 9–45)

These route modules exist and are wired into the router, but serve stub responses:

- Phase 9: Governance & compliance
- Phase 10: AI platform
- Phase 11: Platform operations
- Phase 12: Marketplace
- Phase 13: Enterprise analytics
- Phase 14: Enterprise production
- Phase 15: Post-GA LTS
- Phases 16–45: Enterprise intelligence through v7 GA

Each phase has corresponding seed scripts, test files, and documentation, but no real business logic beyond the repository scaffold pattern.

5. Infrastructure Assessment

5.1 Current Deployment

Component	Platform	Plan	Limitations
API	Render	Free	Spins down after idle, 50s+ cold start

Component	Platform	Plan	Limitations
Database	Supabase	Free	500 MB, pauses after 7 days inactivity
Web	Cloudflare Workers	Free	Static export, good performance
Keep-alive	GitHub Actions	—	Pings Supabase every 5 days

5.2 Infrastructure Risks

- 1 Cold start latency — Free Render instance needs 50+ seconds to wake. First request after idle will timeout for most clients.
- 2 Database pause — Supabase free tier pauses after 7 days. Keep-alive workflow mitigates but doesn't eliminate risk.
- 3 No redundancy — Single instance, no health check failover beyond Render's built-in restart.
- 4 No staging environment — All changes deploy directly to production on main branch push.
- 5 Migration coupling — Migrations run at startup (npm run migrate && npm start). Failed migration blocks API.
- 6 No secrets rotation — JWT_SECRET and PORTAL_TOKEN_SECRET are static generated values.

5.3 Build Pipeline

- CI workflow runs on push to main
- Build: npm ci --omit=dev (API), npm ci && npm run build (Web)
- Web static export (NEXT_OUTPUT=export) for Cloudflare Workers deployment
- No automated test execution in CI (tests exist but aren't gated)
- 16 GitHub Actions workflows (most are release governance templates, not actively triggered)

6. Code Quality Assessment

6.1 Strengths

- Consistent architecture — Repository pattern with clear separation (routes → services → repositories)
- Comprehensive middleware — Auth, CORS, rate limiting, body limits, security headers, audit
- Multi-tenant design — Clean tenant isolation from day one
- Extensive documentation — Sprint/phase docs provide full traceability for future features
- API completeness — 177 route modules cover broad functionality

6.2 Technical Debt

- 1 No framework — Raw Node.js HTTP handling in router.js (1,093 lines of if-statements). Fragile, hard to maintain, no middleware composition.
- 2 Mixed TypeScript/JavaScript — API is predominantly .js with some .ts files. Web is TypeScript. Inconsistent typing.
- 3 Massive router — Single-file router with all route matching. No modular route registration.
- 4 Test execution gap — 770 test files exist but no CI gating. Unknown pass rate.

- 5 Script sprawl — 702 scripts in scripts/ directory, most are seed files for scaffolded phases.
- 6 Migration count mismatch — Only 7 actual SQL files despite codebase suggesting 773 migrations (numbered from naming convention, not actual files).
- 7 Stub routes consuming resources — Phases 9–45 route modules are loaded and evaluated on every request even though they serve no production function.
- 8 No error monitoring — No Sentry, DataDog, or equivalent. Errors disappear into Render logs.
- 9 Frontend component density — Only 44 TSX components serve the entire application. Individual components are likely very large.
- 10 No dependency lockfile at root — Root package.json has only 3 production dependencies but no package-lock.json enforcement visible.

6.3 Security Posture

Control	Status
JWT authentication	■ Implemented
Permission-based authorization (RBAC)	■ Implemented
CORS restriction	■ Configured per-environment
Rate limiting	■ Middleware in place
Security headers	■ Applied globally
Input validation	■■ Route validation middleware exists, coverage unknown
SQL injection prevention	■■■ Uses pg parameterized queries (assumed)
Secret management	■■■ Static secrets, no rotation
HTTPS enforcement	■ Via Render/Cloudflare
Audit logging	■ Request audit middleware
Dependency scanning	■ Not gated in CI
Penetration testing	■ Not performed

7. Frontend Assessment

7.1 Technology

- Framework: Next.js 15.5.20 with React 19.1.1
- Deployment: Static export to Cloudflare Workers (no SSR)
- Styling: CSS files (globals.css, storefront.css, mobile-fixes.css, whitespace-modern.css, storefront-builder.css)
- State: No external state library (React state + fetch)
- Auth: Custom authFetch wrapper in src/lib/api.ts

7.2 Component Inventory (44 TSX files)

Key workspace components: StorefrontBuilder, PublicStorefront, CustomerWorkspace, DispatchBoard, CrmPipeline, ProfileWorkspace, PlatformAdminWorkspace, TeamManagementWorkspace, PasswordInput

7.3 Frontend Issues

- 1 Component size — Large monolithic components (StorefrontBuilder, PublicStorefront likely 1000+ lines each)
- 2 No component library — No shared design system, no Tailwind/Chakra/MUI
- 3 CSS specificity wars — Multiple CSS files with !important overrides to fix font sizes
- 4 No testing — No Jest/Vitest/Playwright configured for frontend
- 5 No code splitting — Static export means no dynamic imports benefit

8. Database Assessment

8.1 Schema Management

- PostgreSQL 16 on Supabase (free tier, oregon region)
- 7 migration SQL files (latest: 773_general_availability_cutover.sql)
- Migrations run idempotently at API startup
- Schema-per-tenant isolation

8.2 Database Risks

- 1 Free tier limits — 500 MB storage, no point-in-time recovery, no backups
- 2 Pause risk — Project pauses after 7 days inactivity
- 3 No connection pooling — Direct connection via Supabase pooler (port 5432)
- 4 Single region — ca-central-1, no read replicas
- 5 No database monitoring — No pg_stat alerts, no slow query tracking

9. CI/CD Assessment

9.1 Active Workflows

Workflow	Purpose	Status
ci.yml	Build verification	Active
docker-build.yml	Container image	Available
supabase-keepalive.yml	Prevent DB pause	Active (every 5 days)
build-release.yml	Release packaging	Template
online-smoke.yml	Post-deploy smoke	Available

9.2 CI Gaps

- 1 No test gating — Tests don't run in CI, no coverage threshold
- 2 No lint step — No ESLint/Prettier enforcement

- 3 No type checking — TypeScript errors don't block deploy
- 4 No security scanning — No npm audit, no SAST tools
- 5 No preview deployments — No PR preview environments
- 6 Direct-to-main — No branch protection, no required reviews

10. Scalability Assessment

10.1 Current Capacity

- API: Single free-tier Render instance (limited RAM/CPU)
- Database: 500 MB Supabase free tier
- Tenants: 2 active workspaces
- Users: Single admin user

10.2 Scaling Bottlenecks

- 1 Single instance — No horizontal scaling on free tier
- 2 Startup time — Cold starts take 50+ seconds (migration + dependency load)
- 3 Router complexity — $O(n)$ route matching through if-chain (~200 comparisons per request)
- 4 No caching — No Redis, no CDN caching for API responses
- 5 No background jobs — Long-running tasks block request threads

11. Recommendations Summary

Critical (Address Immediately)

- 1 Upgrade Render to at least Starter plan (\$7/mo) to eliminate cold starts
- 2 Enable test gating in CI — even running existing tests would catch regressions
- 3 Add error monitoring (Sentry free tier)
- 4 Add TypeScript checking to CI build

High Priority (Next 30 Days)

- 1 Refactor router.js — extract route groups into modular route files with proper middleware composition
- 2 Remove or lazy-load Phase 9–45 stub modules to reduce startup time
- 3 Add branch protection to main
- 4 Set up staging environment on Render
- 5 Pin all root dependencies and add package-lock.json
- 6 Add Playwright E2E tests for critical paths (login, create job, storefront)

Medium Priority (Next 90 Days)

- 1 Migrate API to Express or Fastify for proper middleware, error handling, and routing
- 2 Standardize on TypeScript across the API
- 3 Implement connection pooling with PgBouncer
- 4 Break large TSX components into sub-components
- 5 Add shared UI component library (Radix/shadcn or similar)
- 6 Set up automated database backups

Low Priority (Backlog)

- 1 Evaluate multi-app architecture (per spec in [.kiro/specs/multi-app-architecture/](#))
- 2 Implement feature flags for phase-gating
- 3 Add OpenTelemetry distributed tracing
- 4 Progressive deployment (canary/blue-green)